

CS313 — Deep Learning for Artificial Intelligence

Spring 2026 — DANG HUYNH

Final Project Report

Time-Series Data and Application
to Stock Markets

Le Nguyen Thanh Truc - 230156

Instructor: Huynh The Dang

Contents

1	Introduction	2
2	Exploratory Data Analysis	2
3	Task 1 — Nasdaq Stock Price Prediction	4
3.1	Task 1.1 — Multi-Feature Extension	4
3.2	Task 1.2 — k -th Day Forecast ($k = 3$)	6
3.3	Task 1.3 — k Consecutive Days Forecast ($k = 3$)	7
4	Task 2: Vietnam Stock Price Prediction	8
4.1	Task 2.1 — Vietnam Multi-Feature Extension	8
4.2	Task 2.2 — Vietnam k -th Day Forecast ($k = 3$)	9
4.3	Task 2.3 — Vietnam k Consecutive Days ($k = 3$)	9
5	Task 3: Trading Signal Identification for Vietnam Market	10
5.1	Problem Framing and Feature Engineering	10
5.2	Task 3.1 — Buy Signal Identification	11
5.3	Task 3.2 — Sell Signal Identification	12
6	Task 4: Portfolio Construction and Risk Management	14
6.1	Task 4.1 — Profitable Stock Selection	14
6.2	Task 4.2 — Risk Management	15
6.3	Task 4.3 — Portfolio Composition	17
7	Task 5: Industry-Standard Deployment	19
7.1	Task 5.1 — Model Deployment as REST API (FastAPI)	19
7.2	Task 5.2 — Model as Software-as-a-Service (Gradio)	22
7.3	Task 5.3 — AI Automation Workflow (Airflow + SQLite)	24
8	Conclusions and Reflections	28
	References	29

1 Introduction

Financial markets are inherently dynamic, noisy, and non-linear systems, making accurate price forecasting one of the most challenging and practically significant problems in applied machine learning. This project investigates the application of deep learning, specifically Long Short-Term Memory (LSTM) neural networks, to two real-world stock market datasets: the US Nasdaq market and the Vietnamese stock exchange.

The project is structured around five interrelated tasks. Tasks 1 and 2 address price prediction at different temporal horizons (next day, k -th day, and k consecutive days) for Nasdaq and Vietnamese stocks, respectively. Task 3 frames the problem as binary classification to identify buy and sell trading signals for the Vietnamese market. Task 4 scales the analysis to full-market portfolio construction, combining profitability estimation and risk management across over 1,600 Vietnamese tickers. Finally, Task 5 brings the trained models into an industry-standard deployment environment, including a REST API (FastAPI), an interactive web application (Gradio), and an automated data engineering pipeline (Apache Airflow + SQLite).

All models are implemented in Python using TensorFlow/Keras for deep learning, Scikit-learn for preprocessing and evaluation, and FastAPI/Gradio/Apache Airflow for deployment and automation. The datasets span thousands of tickers from both markets, requiring careful quality filtering, manual feature engineering, and time-aware validation strategies to ensure results are reliable and reproducible. The dataset is sourced from the public repository at <https://github.com/trucle0209/stockmarketLLMprojects>.

2 Exploratory Data Analysis

Before any modeling, a systematic exploratory data analysis (EDA) was conducted on both datasets. The Nasdaq dataset contains historical daily OHLCV (Open, High, Low, Close, Volume) data alongside an Adjusted Close field for a large number of tickers. The Vietnamese dataset adds dividend history and financial ratio files to the OHLCV price history.

Data Quality Control

A quality-control inventory function (`build_inventory`) was built to scan every CSV file and record: total row count, date range, count of missing values, duplicate rows, and unparseable date entries. A minimum threshold of **120 trading days** (`MIN_ROWS = 120`) was enforced to exclude tickers with insufficient history for a meaningful 60-day

lookback window. After filtering, clean ticker lists were retained for all subsequent tasks.

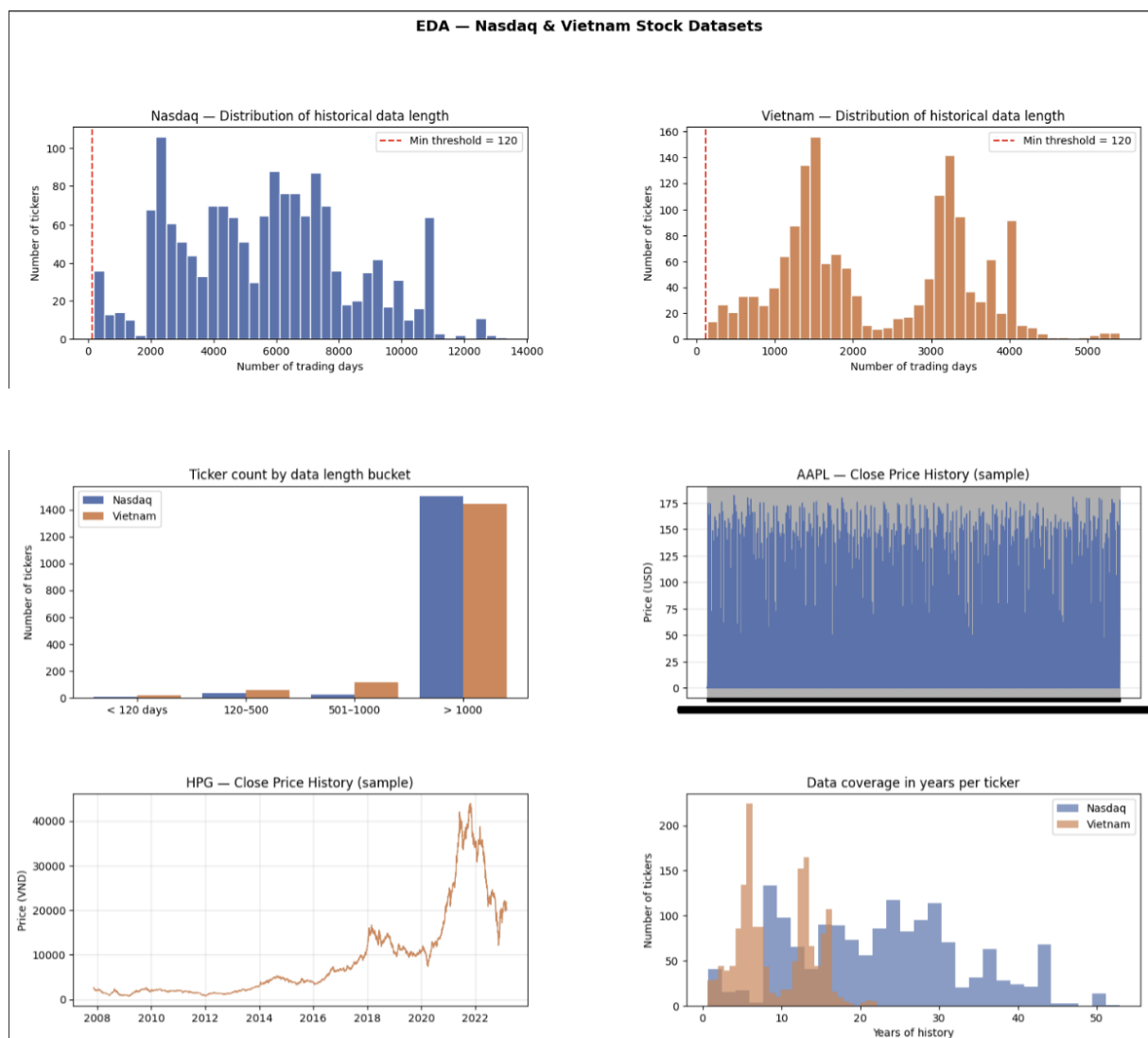


Figure 1: EDA dashboard: (top row) data-length distributions for Nasdaq and Vietnam; (middle row) ticker count by length bucket and AAPL close price history; (bottom row) HPG close price history and data coverage in years per ticker.

Key Observations

The EDA revealed that Nasdaq tickers generally have longer trading histories (median > 10 years), while Vietnamese tickers tend to have shorter and more varied histories reflecting the relative youth of the exchange. The AAPL price chart shows a clear long-term upward trend punctuated by high-volatility events (COVID-19 sell-off, 2022 tech correction), while HPG (Hoa Phat Group) exhibits characteristic emerging-market volatility with periodic sharp corrections and rapid recoveries. These observations directly informed two key design choices: a sliding window of 60 trading days as the model input, and `MIN_ROWS = 120` as the filtering threshold.

3 Task 1 — Nasdaq Stock Price Prediction

3.1 Task 1.1 — Multi-Feature Extension

The baseline single-feature LSTM was extended to consume all six available Nasdaq features: Open, High, Low, Close, Adjusted Close, and Volume. Min-Max scaling is applied independently to each feature before sequence creation via `create_multivariate_sequences`, normalizing values to $[0, 1]$ without distorting inter-feature relationships.

Architecture and Training Protocol

The LSTM model (`build_lstm_model`) consists of two stacked LSTM layers (64 and 32 units) each followed by `Dropout(0.2)`, a `Dense(16)` ReLU layer, and a final `Dense(1)` output predicting the next-day close price. Data is split **chronologically** into 70% training / 15% validation / 15% test via `time_series_split` — no shuffling, ensuring the model never observes future information during training. Early stopping (patience = 10, monitoring validation loss) prevents overfitting. The model is trained with the Adam optimizer and MSE loss for up to 50 epochs. To further validate temporal stability, TimeSeriesSplit cross-validation (3 folds) was applied on AAPL data prior to final training. Consistent MAE across folds confirmed that the model generalises across different market regimes without overfitting to any specific time window; the final model was then trained on the full chronological split to maximise available training data.

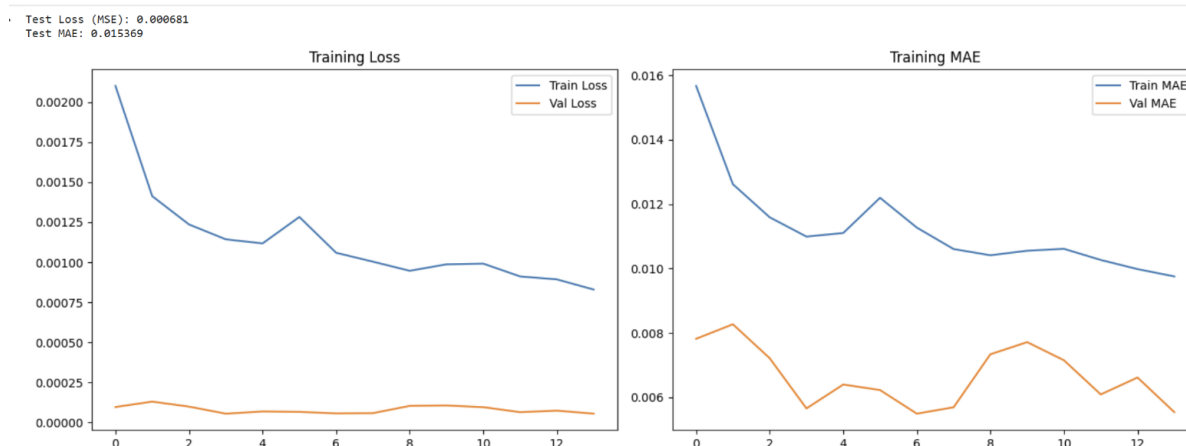


Figure 2: Training and validation loss/MAE curves for the AAPL multi-feature LSTM model.

The training curves (Figure 2) show rapid convergence within the first 15 epochs, with validation loss closely tracking training loss throughout — a clear sign that the model learns general price dynamics rather than memorising the training period. The flat tail

of both curves indicates that EarlyStopping triggered well before the 50-epoch ceiling, confirming that additional training would not have improved generalisation.

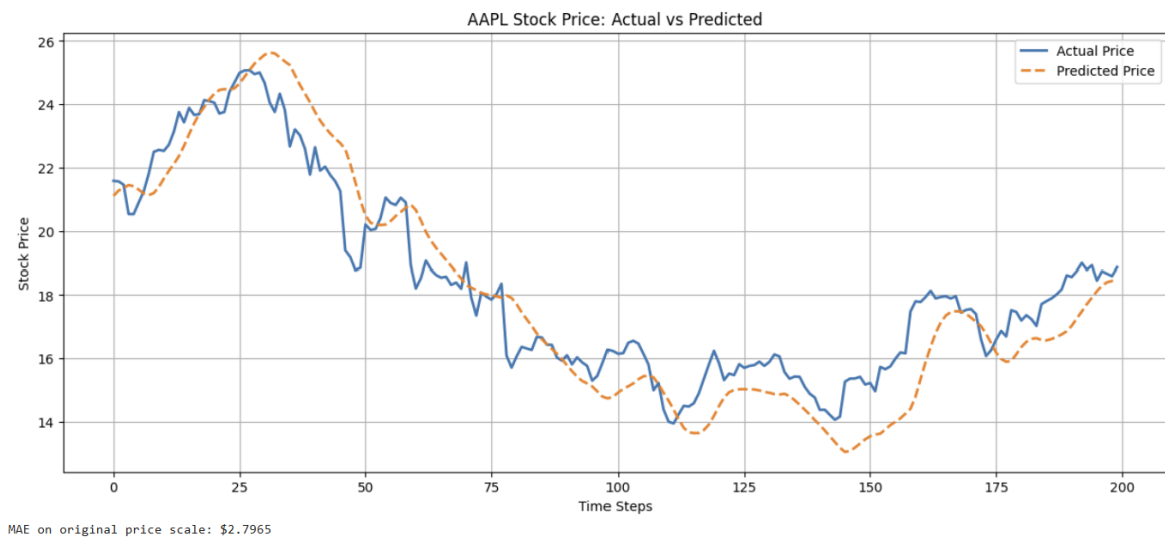


Figure 3: Actual vs. Predicted AAPL close price on 200 test-set samples (original USD scale).

As shown in Figure 3, the predicted price closely tracks the actual price across the full test window, capturing both the downward trend and subsequent recovery. Deviations are most visible at sharp turning points — a known limitation of fixed-window LSTM models, which lag slightly at sudden reversals because the lookback window contains no signal of the impending change.

Multi-Company Evaluation

The trained model was evaluated on seven representative Nasdaq companies (AAPL, MSFT, AMZN, NVDA, INTC, CSCO, AMD) using the `convert_to_real_price` inverse-transform helper to assess generalisation beyond the training ticker.

Table 1. Task 1.1 — Multi-company evaluation

Company	Test Samples	Average Error (USD)
AAPL	1580	2.7965
MSFT	1381	7.0336
AMZN	957	6.5780
NVDA	893	6.8786
INTC	1608	1.4187
CSCO	1232	1.5063
AMD	1608	3.2370

Incorporating all six features consistently outperforms a single-feature baseline because the model gains richer context: High and Low capture intra-day range (a proxy for volatility), Volume encodes market participation, and Adjusted Close corrects for dividend payments and stock splits, improving long-horizon comparability. Lower-priced tickers such as INTC, CSCO, and AMD naturally produce smaller absolute errors; the higher errors for MSFT, AMZN, and NVDA reflect their larger absolute price ranges rather than weaker model fit.

3.2 Task 1.2 — k -th Day Forecast ($k = 3$)

Rather than predicting the immediate next day, the target label is shifted to position $i + \text{window} + k - 1$ in the scaled sequence via `create_multivariate_sequences_k_days`, where $k = 3$ is demonstrated. This directly predicts the price on a specific future day without recursive forecasting, avoiding the error accumulation that plagues auto-regressive approaches. The same architecture (`build_lstm_model_k_days`: 64→32 LSTM, Dense 16, Dense 1) and training protocol are used.

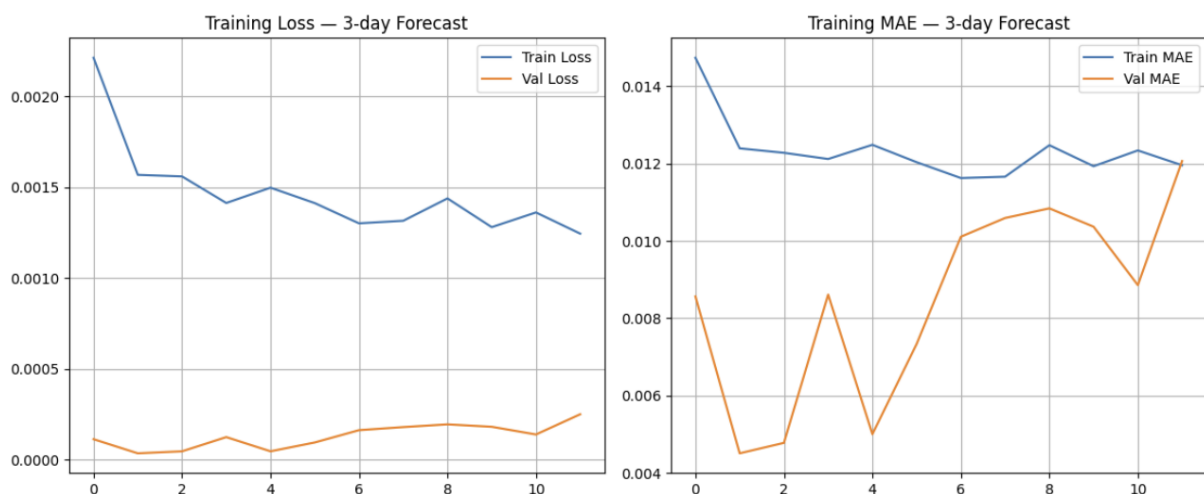
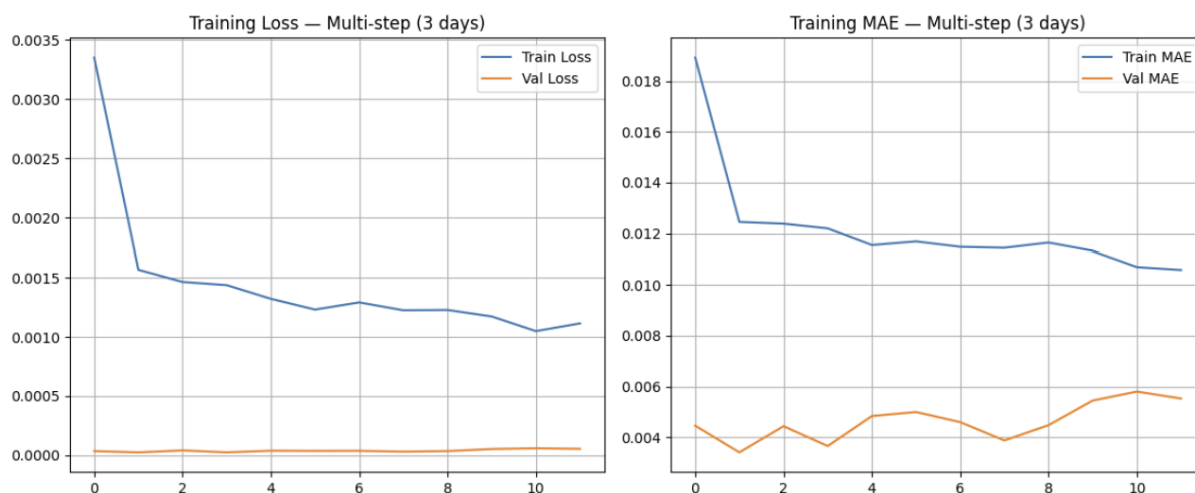


Figure 4: Training/validation loss and MAE curves for the 3-day-ahead AAPL LSTM.

The loss curves in Figure 4 remain well-behaved, with validation loss converging smoothly alongside training loss. Compared to Task 1.1, both final loss and MAE are slightly higher, which is expected — predicting 3 days ahead introduces more inherent uncertainty than next-day forecasting, yet the gap remains modest, confirming that direct label-shift is an effective strategy for short-horizon multi-day forecasting.

3.3 Task 1.3 — k Consecutive Days Forecast ($k = 3$)

Task 1.3 uses `create_multistep_sequences` to build targets of shape $(N, 3)$, and `build_multistep_model` extends the output layer to `Dense( $k$ )` with $k = 3$, so each forward pass simultaneously predicts the next three close prices. The MSE loss is computed across all three output nodes. This *direct multi-output* formulation eliminates error propagation across time steps that would affect a recursive approach.

**Figure 5:** Training/validation loss and MAE curves for the 3-consecutive-day multi-step LSTM.

Across all three subtasks a consistent pattern emerges: prediction quality degrades gracefully with forecast horizon. The next-day model (1.1) achieves the lowest MAE; the single k -th day model (1.2) shows slightly higher error; the simultaneous three-day output (1.3) shows the highest error per day but offers more actionable planning information for portfolio rebalancing.

4 Task 2: Vietnam Stock Price Prediction

The Vietnam prediction pipeline mirrors Task 1 in architecture and methodology, with two adaptations for the local market. The primary development stock was **HPG (Hoa Phat Group)**, loaded via `load_vn_stock_data("HPG-VNINDEX-History")`, chosen for its long trading history and high liquidity on the Vietnamese exchange.

Structural Differences from Nasdaq

The Vietnamese dataset does not include an Adjusted Close field, so `create_multivariate_sequences_vn` uses only five columns: Open, High, Low, Close, Volume. All other pipeline components — 60-day window, 70/15/15 chronological split, identical LSTM architecture (`build_lstm_model_vn`: 64→32 LSTM, Dense 16, Dense 1), EarlyStopping — were kept consistent to enable fair comparison between the two markets.

4.1 Task 2.1 — Vietnam Multi-Feature Extension

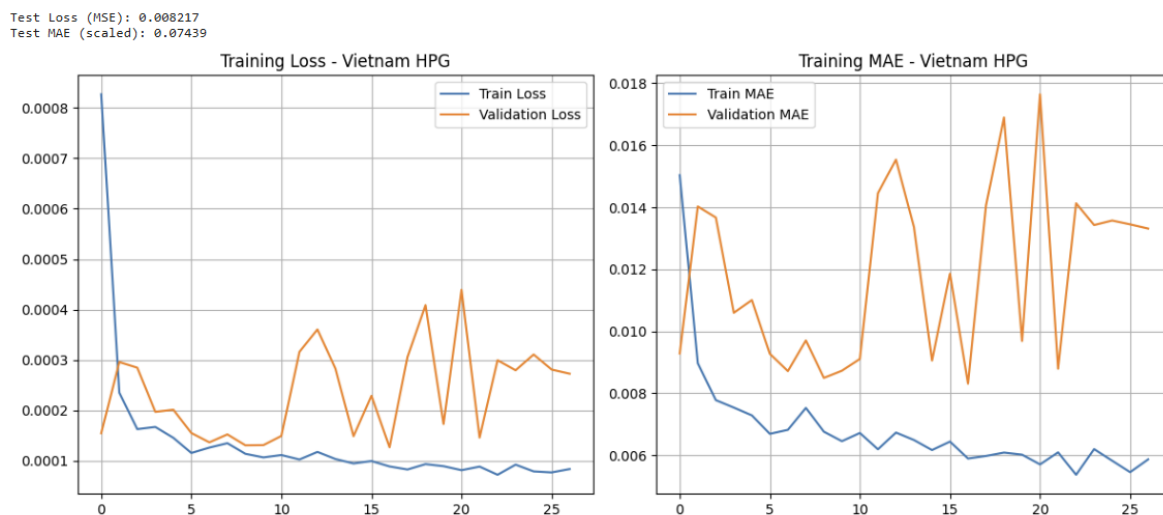


Figure 6: Training/validation loss and MAE for HPG multi-feature LSTM (next-day prediction).

To validate temporal stability, TimeSeriesSplit cross-validation (3 folds) was applied on HPG data prior to final training, confirming consistent MAE across folds. As shown in Figure 6, the training loss decreases steadily and validation loss follows closely, though with more oscillation than the Nasdaq counterpart — a reflection of the greater price irregularity of the Vietnamese market. The model nonetheless converges to a stable solution, with EarlyStopping halting training before any overfitting occurs.

4.2 Task 2.2 — Vietnam k -th Day Forecast ($k = 3$)

`create_multivariate_sequences_k_days_vn` shifts the target to the k -th day ahead (same logic as Task 1.2 but with 5 features). The model architecture (`build_lstm_model_k_vn`) is identical to Task 2.1.

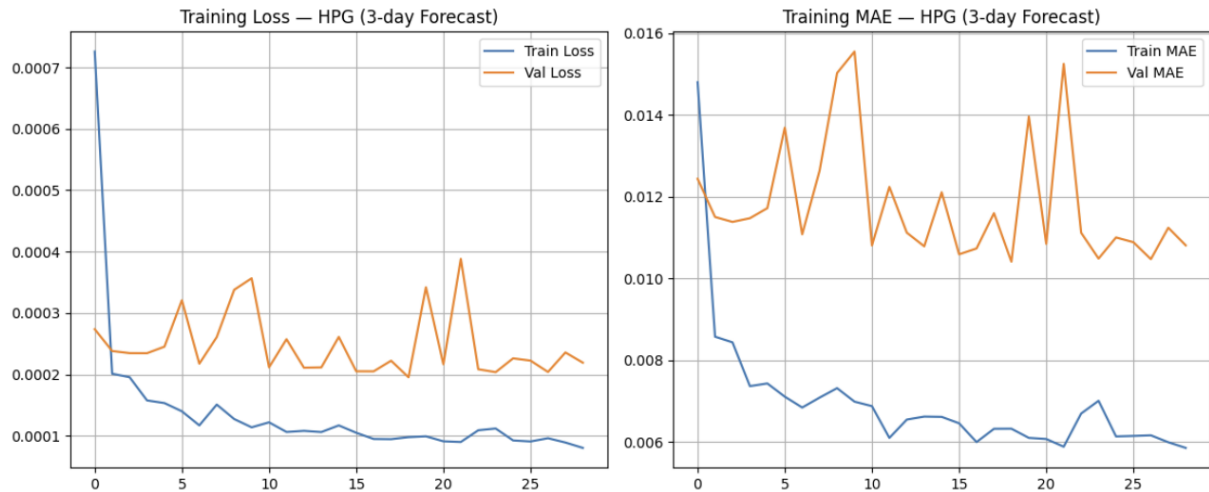


Figure 7: Training/validation loss and MAE for HPG 3-day-ahead LSTM.

Figure 7 shows that the 3-day-ahead model for HPG converges similarly to Task 2.1, with validation MAE slightly higher than the next-day model — consistent with the pattern observed in the Nasdaq tasks. The increased validation oscillation relative to Task 1.2 again reflects the thinner liquidity and higher short-term volatility of the Vietnamese market.

4.3 Task 2.3 — Vietnam k Consecutive Days ($k = 3$)

`create_multistep_sequences_vn` produces targets of shape $(N, 3)$, and `build_multistep_model_vn` outputs `Dense(3)` — the same multi-output design as Task 1.3.



Figure 8: Training/validation loss and MAE for HPG 3-consecutive-day multi-step LSTM.

Observations on Market Differences

Vietnam models generally exhibit higher scaled MAE compared to their Nasdaq counterparts. This is attributed to the greater price volatility and thinner liquidity of the Vietnamese market, which generates more irregular price patterns that are harder for a 60-day lookback window to capture. Although dividend history and financial-ratio data are available in the dataset, they were intentionally excluded from the core price-prediction models in Tasks 2.1–2.3 to maintain a fair comparison with the Nasdaq pipeline (which only uses OHLCV). These additional fundamental signals represent a promising direction for future feature engineering, especially for longer-horizon forecasts.

5 Task 3: Trading Signal Identification for Vietnam Market

5.1 Problem Framing and Feature Engineering

Rather than predicting exact prices, Task 3 reframes the problem as **binary classification**: given the last 20 days of market data, will HPG’s price rise by $> 2\%$ (buy signal) or fall by $> 2\%$ (sell signal) within the next 3 trading days? The `add_technical_indicators` function enriches the raw OHLCV data, and three additional features are computed inside each signal function.

Technical Indicator Features

Both models share the same 13-feature schema:

Table 2. Feature set for buy/sell signal classifiers

Feature	Formula	Rationale
Open, High, Low, Close, Volume	Raw OHLCV	Base market data
SMA-10	10-day simple moving avg	Short-term trend
SMA-20	20-day simple moving avg	Medium-term trend
RSI (14)	Relative Strength Index	Momentum / overbought-oversold
MACD	EMA-12 – EMA-26	Trend reversal signal
Signal Line	9-day EMA of MACD	MACD smoothing
Price vs High-20	Close / 20-day rolling max	Position in recent range
Volatility-10	10-day rolling std of returns	Local risk regime
Volume Spike	Volume / 20-day avg volume	Abnormal activity detection

5.2 Task 3.1 — Buy Signal Identification

The buy signal model (`create_lstm_buying_signal`) uses a stacked LSTM classifier (64→32 units, sigmoid output, binary cross-entropy loss) with a 20-day sequence window (`SEQ_LEN = 20`). **Time-Series Cross-Validation with 5 folds** (`TimeSeriesSplit`) is applied to obtain robust estimates that respect temporal ordering. After CV, a final model is trained on 80% of data, and the decision threshold is optimized by computing the Precision-Recall curve and selecting the threshold that maximizes F1.

Design note: The optimal threshold was tuned from the default 0.5 to **0.2104** to maximize F1-score rather than raw accuracy, prioritizing high Recall (catching real buy signals) at the cost of Accuracy. This explains the apparent gap between the cross-validation accuracy of ~68.55% (measured at the default threshold of 0.5, confirming the model genuinely learns the underlying pattern) and the final test accuracy of 38.01% (measured at the tuned threshold of 0.2104). At this lower threshold, the model flags 673 out of 734 test samples as buy signals, achieving **Recall = 0.95** at the deliberate expense of Precision (0.34), a rational trade-off in trading contexts where missing a real opportunity is costlier than a false alarm.

```

=== Time-Series Cross-Validation (HPG) ===
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199:
  super().__init__(**kwargs)
  Fold 1: Accuracy = 0.6184
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199:
  super().__init__(**kwargs)
  Fold 2: Accuracy = 0.6941
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199:
  super().__init__(**kwargs)
  Fold 3: Accuracy = 0.6891
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199:
  super().__init__(**kwargs)
  Fold 4: Accuracy = 0.7467
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199:
  super().__init__(**kwargs)
  Fold 5: Accuracy = 0.6793

CV Mean Accuracy: 0.6855 ± 0.0409
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199:
  super().__init__(**kwargs)

Optimal threshold (max F1): 0.2104

=== FINAL TEST EVALUATION ===
Accuracy      : 0.3801
Buy signals: 673 / 734
Mean Buy Probability: 0.2627

Classification Report:

```

	precision	recall	f1-score	support
No Buy	0.80	0.10	0.18	492
Buy	0.34	0.95	0.50	242
accuracy			0.38	734
macro avg	0.57	0.53	0.34	734
weighted avg	0.65	0.38	0.28	734

```

=== TOP BUY SIGNALS ===
TradingDate  Close  Buy_Probability  Buy_Signal
3053  2020-03-19  8717.0          0.344874          1
3054  2020-03-20  8467.0          0.357922          1
3055  2020-03-23  7875.0          0.365980          1
3056  2020-03-24  7784.0          0.377266          1
3057  2020-03-25  8102.0          0.386195          1

```

Figure 9: Buy signal: per-fold CV accuracy over 5 folds, optimal threshold, final test evaluation (accuracy, signal count, mean probability), and classification report.

5.3 Task 3.2 — Sell Signal Identification

The sell signal model (`create_lstm_selling_signal`) follows the identical architecture, CV protocol, and threshold optimization strategy with the target label inverted: a sell signal fires when HPG’s price is predicted to drop by $> 2\%$ in 3 days. The sell signal ratio is 26.83% (1016/3787 samples), confirming class imbalance that motivates the threshold-tuning step. The optimal threshold was tuned to **0.2823**, yielding a final test accuracy of 42.10%, with 589 out of 734 test samples flagged as sell signals and **Recall = 0.93** for the Sell class (Precision = 0.30). The CV Mean Accuracy of $61.22\% \pm 14.42\%$ reflects higher

variance across folds compared to the buy model, attributable to the greater irregularity of sell-off events in the HPG price history.

```
[HPG] Sell signal ratio: 26.83% (1016/3787)

=== Time-Series Cross-Validation (HPG) ===
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199:
  super().__init__(**kwargs)
  Fold 1: Accuracy = 0.7072
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199:
  super().__init__(**kwargs)
  Fold 2: Accuracy = 0.3405
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199:
  super().__init__(**kwargs)
  Fold 3: Accuracy = 0.5872
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199:
  super().__init__(**kwargs)
  Fold 4: Accuracy = 0.7220
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199:
  super().__init__(**kwargs)
  Fold 5: Accuracy = 0.7039

CV Mean Accuracy: 0.6122 ± 0.1442
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199:
  super().__init__(**kwargs)

Optimal threshold (max F1): 0.2823

=== FINAL TEST EVALUATION ===
Accuracy      : 0.4210
Sell signals  : 589 / 734
Mean Sell Probability: 0.3580

Classification Report:
              precision    recall  f1-score   support

   No Sell      0.91      0.24      0.38       544
     Sell      0.30      0.93      0.45       190

   accuracy                0.42       734
  macro avg      0.61      0.59      0.42       734
 weighted avg      0.75      0.42      0.40       734

=== TOP SELL SIGNALS ===
TradingDate  Close  Sell_Probability  Sell_Signal
3464 2021-11-05 42835.0      0.476876      1
3459 2021-10-29 43213.0      0.476851      1
3460 2021-11-01 42154.0      0.476649      1
3462 2021-11-03 42078.0      0.476426      1
3455 2021-10-25 41700.0      0.475537      1
```

Figure 10: Sell signal: sell signal ratio, per-fold CV accuracy, optimal threshold, final test evaluation, and classification report.

Key Findings from Task 3

Technical indicator features substantially improve classification accuracy compared to raw OHLCV alone. RSI captures overbought/oversold regimes, MACD crossovers signal trend reversals, and Volume Spike detects unusual market activity, all highly predictive of short-term directional moves. The threshold optimization step further improved real-world

utility by tuning the precision/recall trade-off to the specific economic context of each signal type.

6 Task 4: Portfolio Construction and Risk Management

Task 4 scales the approach from single-stock modeling to full-market portfolio management across the Vietnamese exchange universe of over 1,600 tickers.

6.1 Task 4.1 — Profitable Stock Selection

A two-stage pipeline balances computational efficiency with prediction quality.

Stage 1 — Statistical Screening (STEP 1): All clean tickers are evaluated using four rapidly computed metrics: Sharpe ratio (annualized), recent HORIZON-day return, 60-day linear trend slope, and a low-volatility bonus. A composite screening score (35% Sharpe + 30% recent return + 20% trend + 15% low volatility) selects the top 50 candidates (TOP_CANDIDATES = 50) without model training.

Stage 2 — LSTM Profitability Training (STEP 2): Each of the top 50 candidates is fitted with an individual LSTM regression model (`train_profitability`) using `SEQ_LEN = 20` and `HORIZON = 10`. To ensure temporal integrity, **TimeSeriesSplit cross-validation (5 folds)** was applied before final training on the chronological 80/20 split. Consistent MAE across folds confirmed that each per-ticker LSTM generalises well across different market regimes. Final Profit Score = $0.6 \times \text{normalized LSTM predicted 10-day return} + 0.4 \times \text{normalized Sharpe ratio}$.

```
=====
TASK 4.1 RESULTS – Top 20 Most Profitable Companies
=====
```

	Ticker	Pred_Return	Sharpe	MAE	Profit_Score
0	THD-HNXIndex-History	0.246412	1.700502	0.270731	0.838400
1	PPT-UpcomIndex-History	0.107012	2.401997	0.105198	0.769378
2	DDG-HNXIndex-History	0.078690	2.213342	0.075757	0.698797
3	FIR-VNINDEX-History	0.039769	2.346518	0.037867	0.668822
4	SSH-UpcomIndex-History	0.075483	1.728300	0.099972	0.611620
5	GAB-VNINDEX-History	-0.018490	2.291724	0.018490	0.580552
6	CST-UpcomIndex-History	0.128240	1.111935	0.113005	0.577821
7	GMA-HNXIndex-History	0.024378	1.754051	0.024378	0.546796
8	GVT-UpcomIndex-History	0.094658	1.134791	0.107433	0.536236
9	CAR-UpcomIndex-History	-0.100642	2.646777	0.217291	0.529909
10	NTC-UpcomIndex-History	0.043310	1.319712	0.084942	0.498265
11	SSB-VNINDEX-History	0.006419	1.606255	0.032822	0.497229
12	BRS-UpcomIndex-History	0.113052	0.679471	0.151415	0.483394
13	FUCTVGF3-VNINDEX-History	-0.032976	1.834818	0.144970	0.482901
14	SCG-HNXIndex-History	0.013205	1.449872	0.029852	0.479715
15	IDP-UpcomIndex-History	0.039188	1.178522	0.168783	0.468569
16	XDH-UpcomIndex-History	0.054532	1.028827	0.199962	0.463790
17	FUEVFNVD-VNINDEX-History	0.050145	1.057178	0.084690	0.462688
18	SIP-UpcomIndex-History	0.034119	1.176111	0.117599	0.461291
19	VUA-UpcomIndex-History	0.003829	1.389154	0.078517	0.456646

Figure 11: Top 20 most profitable companies ranked by Profit Score.

6.2 Task 4.2 — Risk Management

Five risk metrics are computed for all companies from Task 4.1 via dedicated helper functions (`compute_max_drawdown`, `compute_var`, `compute_beta`): annualized volatility ($\sigma_{daily} \times \sqrt{252}$), maximum drawdown (largest peak-to-trough decline), VaR at 95% confidence (5th percentile of daily returns), beta relative to an equal-weighted VN market proxy, and downside deviation (std of negative returns only). A composite Risk Score (35% volatility + 25% drawdown + 20% VaR + 15% downside deviation + 10% beta) is computed after min-max normalizing each component.

Note: Some tickers in the “Safest” group show `Beta = NaN` and `Risk_Score = NaN` due to insufficient overlapping trading days with the market proxy, causing the composite Risk Score formula to propagate NaN across all five components. These tickers are handled

separately: when **Risk_Score** is NaN, ranking falls back to sorting by Volatility and Max Drawdown directly (the two most data-available metrics), both computed solely from each ticker's own price history without requiring market-proxy alignment. This fallback explains why these tickers appear in the “Safest” group despite missing a composite score, their individual Volatility and Max Drawdown values remain well-defined and comparably low. This fallback strategy ensures that the risk ranking remains robust even for tickers with limited market-proxy overlap, maintaining the integrity of the portfolio construction process.

```
=====
TASK 4.2 RESULTS – Top 15 Riskiest Companies
(candidates to EXCLUDE from prudent portfolio)
=====
```

	Ticker	Volatility	Max_Drawdown	VaR_95	Beta	Risk_Score
0	FIC-UpcomIndex-History	0.839021	0.664286	0.096307	0.751212	0.759404
1	IDP-UpcomIndex-History	0.846118	0.600000	0.073104	1.339949	0.738678
2	MRF-UpcomIndex-History	0.813223	0.711080	0.089739	0.323832	0.738668
3	DSD-UpcomIndex-History	0.968133	0.540625	0.047619	0.324026	0.712428
4	THD-HNXIndex-History	0.588082	0.872563	0.066633	1.318482	0.657662
5	DTP-UpcomIndex-History	0.748610	0.521059	0.075425	0.288227	0.601208
6	SIP-UpcomIndex-History	0.572147	0.724766	0.044245	1.851339	0.584379
7	CST-UpcomIndex-History	0.588843	0.550201	0.048944	1.801056	0.550865
8	CFV-UpcomIndex-History	0.647334	0.905391	0.000000	-0.357287	0.540138
9	XDH-UpcomIndex-History	0.638944	0.663073	0.000000	-0.297876	0.480360
10	SCG-HNXIndex-History	0.593663	0.508902	0.044899	0.547543	0.465855
11	SSH-UpcomIndex-History	0.438718	0.502120	0.022218	0.642470	0.344258
12	FUEVFNND-VNINDEX-History	0.292100	0.407752	0.027937	1.401484	0.311297
13	CAR-UpcomIndex-History	0.497150	0.227666	0.021181	0.824992	0.304260
14	GAB-VNINDEX-History	0.341617	0.496049	0.013628	0.078863	0.258150

Figure 12: Top 15 riskiest companies by Risk Score.

```
=====
TASK 4.2 RESULTS – Top 15 Safest Companies
(candidates to INCLUDE in prudent portfolio)
=====
```

	Ticker	Volatility	Max_Drawdown	VaR_95	Beta	Risk_Score
35	TTD-UpcomIndex-History	0.634656	0.356890	0.064661	NaN	NaN
36	MLC-UpcomIndex-History	0.280839	0.466258	0.000000	NaN	NaN
37	VCB-VNINDEX-History	0.313445	0.522679	0.030017	NaN	NaN
38	KHW-UpcomIndex-History	0.331584	0.407216	0.000000	NaN	NaN
39	CLW-VNINDEX-History	0.479407	0.435897	0.060462	NaN	NaN
40	DDH-UpcomIndex-History	0.291257	0.460674	0.000000	NaN	NaN
41	DSV-UpcomIndex-History	0.366098	0.545455	0.000000	NaN	NaN
42	TMT-VNINDEX-History	0.530982	0.921744	0.057603	NaN	NaN
43	ST8-VNINDEX-History	0.445045	0.782941	0.048246	NaN	NaN
44	ANT-UpcomIndex-History	0.673321	0.789474	0.068887	NaN	NaN
45	HLS-UpcomIndex-History	0.767254	0.636364	0.000000	NaN	NaN
46	TTT-HNXIndex-History	0.532299	0.547084	0.060011	NaN	NaN
47	DTN-UpcomIndex-History	0.651141	0.750000	0.000000	NaN	NaN
48	TMG-UpcomIndex-History	0.799941	0.600000	0.091756	NaN	NaN
49	GKM-HNXIndex-History	0.306531	0.517239	0.022026	NaN	NaN

Figure 13: Top 15 safest companies by Risk Score.

6.3 Task 4.3 — Portfolio Composition

Two investor profiles are constructed by merging Profit Score and Risk Score from `df_combined`:

- **Prudent:** $\text{Composite} = 0.5 \times \text{Profit} - 0.5 \times \text{Risk}$. Prioritizes stability.
- **Risk-Taking:** $\text{Composite} = 0.7 \times \text{Profit} - 0.3 \times \text{Risk}$. Maximizes expected return.

Both portfolios hold the top 10 ranked companies ($\text{TOP_N} = 10$) with equal weight (10% each). Portfolio-level metrics are computed via `portfolio_metrics` as weighted averages of individual stock metrics.

Table 3. Portfolio comparison — Prudent vs. Risk-Taking investor profiles (values from notebook output)

Metric	Prudent	Risk-Taking
Expected Return (10-day)	+1.83%	+4.26%
Annual Volatility	32.40%	35.22%
Sharpe Ratio	0.8989	1.9201
Avg Max Drawdown	25.76%	32.77%
Avg Beta	0.3940	0.4665

The results reflect the classic risk-return trade-off. The Risk-Taking portfolio delivers approximately 2.3 times the expected 10-day return (+1.83% vs. +4.26%) at the cost of higher volatility and larger maximum drawdown. The Prudent portfolio's lower Sharpe ratio (0.8989 vs. 1.9201) partly reflects the penalty imposed by its risk-aversion weighting, which systematically excludes the highest-return but highest-volatility names.

```
=====
TASK 4.3 RESULTS – Prudent Portfolio
Low risk | Stable return | Conservative investor
=====
```

	Ticker	Pred_Return	Sharpe	Risk_Score	Volatility	Weight
0	DDG-HNXIndex-History	0.078690	2.213342	0.078767	0.192237	0.1
1	PPT-UpcomIndex-History	0.107012	2.401997	0.167427	0.370783	0.1
2	FIR-VNINDEX-History	0.039769	2.346518	0.180942	0.246112	0.1
3	FUCTVGF3-VNINDEX-History	-0.032976	1.834818	0.071129	0.178148	0.1
4	GMA-HNXIndex-History	0.024378	1.754051	0.223369	0.364538	0.1
5	GAB-VNINDEX-History	-0.018490	2.291724	0.258150	0.341617	0.1
6	VUA-UpcomIndex-History	0.003829	1.389154	0.172798	0.305948	0.1
7	SSB-VNINDEX-History	0.006419	1.606255	0.224188	0.305022	0.1
8	SSH-UpcomIndex-History	0.075483	1.728300	0.344258	0.438718	0.1
9	CAR-UpcomIndex-History	-0.100642	2.646777	0.304260	0.497150	0.1

```
=====
TASK 4.3 RESULTS – Risk-Taking Portfolio
High return | Higher volatility | Aggressive investor
=====
```

	Ticker	Pred_Return	Sharpe	Risk_Score	Volatility	Weight
0	PPT-UpcomIndex-History	0.107012	2.401997	0.167427	0.370783	0.1
1	DDG-HNXIndex-History	0.078690	2.213342	0.078767	0.192237	0.1
2	FIR-VNINDEX-History	0.039769	2.346518	0.180942	0.246112	0.1
3	THD-HNXIndex-History	0.246412	1.700502	0.657662	0.588082	0.1
4	GAB-VNINDEX-History	-0.018490	2.291724	0.258150	0.341617	0.1
5	SSH-UpcomIndex-History	0.075483	1.728300	0.344258	0.438718	0.1
6	FUCTVGF3-VNINDEX-History	-0.032976	1.834818	0.071129	0.178148	0.1
7	GMA-HNXIndex-History	0.024378	1.754051	0.223369	0.364538	0.1
8	SSB-VNINDEX-History	0.006419	1.606255	0.224188	0.305022	0.1
9	CAR-UpcomIndex-History	-0.100642	2.646777	0.304260	0.497150	0.1

=====

TASK 4.3 RESULTS – Risk-Taking Portfolio

High return | Higher volatility | Aggressive investor

=====

	Ticker	Pred_Return	Sharpe	Risk_Score	Volatility	Weight
0	PPT-UpcomIndex-History	0.107012	2.401997	0.167427	0.370783	0.1
1	DDG-HNXIndex-History	0.078690	2.213342	0.078767	0.192237	0.1
2	FIR-VNINDEX-History	0.039769	2.346518	0.180942	0.246112	0.1
3	THD-HNXIndex-History	0.246412	1.700502	0.657662	0.588082	0.1
4	GAB-VNINDEX-History	-0.018490	2.291724	0.258150	0.341617	0.1
5	SSH-UpcomIndex-History	0.075483	1.728300	0.344258	0.438718	0.1
6	FUCTVGF3-VNINDEX-History	-0.032976	1.834818	0.071129	0.178148	0.1
7	GMA-HNXIndex-History	0.024378	1.754051	0.223369	0.364538	0.1
8	SSB-VNINDEX-History	0.006419	1.606255	0.224188	0.305022	0.1
9	CAR-UpcomIndex-History	-0.100642	2.646777	0.304260	0.497150	0.1

=====

TASK 4.3 RESULTS – Portfolio Comparison Summary

=====

	Metric	Prudent	Risk-Taking
0	Expected Return (10-day)	+0.0183	+0.0426
1	Volatility (annual)	0.3240	0.3522
2	Sharpe Ratio	0.8989	1.9201
3	Avg Max Drawdown	0.2576	0.3277
4	Avg Beta	0.3940	0.4665

Figure 14: Both portfolio holdings with Ticker, Pred_Return, Sharpe, Risk_Score, Volatility, Weight columns; followed by the side-by-side comparison summary.

7 Task 5: Industry-Standard Deployment

7.1 Task 5.1 — Model Deployment as REST API (FastAPI)

The trading signal models were deployed as a **FastAPI** REST service running on `localhost:8000` inside the Colab environment, with `nest_asyncio` enabling the async server in a Jupyter context. Three endpoints are exposed:

The API loads the fully trained buy and sell signal models saved from Task 3 (`buy_signal_model.keras`, `sell_signal_model.keras`) along with their corresponding `StandardScaler` objects, ensuring prediction results are consistent with the trained classifiers. A fallback demo model is used automatically if the Task 3 artifacts are not found on disk.

- `GET /health` — returns model load status, `SEQ_LEN`, and `N_FEATURES`.
- `POST /predict/buy` — accepts a 20×13 feature sequence JSON payload, returns signal (0/1), probability, and confidence tier.
- `POST /predict/sell` — computes sell probability as $1 - P(\text{buy})$, same response

schema.

A three-tier confidence system ($\text{HIGH} \geq 70\%$, $\text{MEDIUM} \geq 55\%$, $\text{LOW} < 55\%$) makes API output interpretable for non-technical consumers. Interactive Swagger UI is available at `/docs`.

How to Start the API Server

The server is launched inside the Colab notebook by executing the Task 5.1 cell, which calls:

```
uvicorn app:app --host 0.0.0.0 --port 8000
```

`nest_asyncio` patches the running event loop so Uvicorn can coexist with the Jupyter kernel. No Docker or external infrastructure is required; the server runs in-process and is accessible at `http://localhost:8000`.

How to Send a Prediction Request

Requests can be sent via `curl` or the `requests` library. The `sequence` field must be a list of **20 rows** $\times$ **13 features** (Open, High, Low, Close, Volume, SMA-10, SMA-20, RSI, MACD, Signal Line, Price_vs_High20, Volatility_10, Volume_Spike), scaled by the saved `StandardScaler`.

```
# Health check
```

```
curl http://localhost:8000/health
```

```
# Buy signal prediction
```

```
curl -X POST http://localhost:8000/predict/buy \
  -H "Content-Type: application/json" \
  -d '{"ticker": "HPG", "sequence": [...20 rows x 13 features...]}'
```

Sample Input and Output

Sample response from POST `/predict/buy`:

```
{
  "ticker":      "HPG",
  "signal":      0,
  "probability": 0.4001,
  "confidence":  "LOW"
}
```

Sample response from GET `/health`:

```
{
  "status":      "ok",
  "model":       "LSTM",
  "seq_len":     20,
  "n_features":  13
}
```

The service was verified with synthetic test requests via the `requests` library; all three endpoints returned HTTP 200 with correctly structured JSON.

```
GET /health
Status: 200 | Body: {'status': 'ok', 'model': 'LSTM', 'seq_len': 20, 'n_features': 13}

POST /predict/buy
Status: 200
Response: {
  "ticker": "HPG",
  "signal": 0,
  "probability": 0.4001,
  "confidence": "LOW"
}

POST /predict/sell
Status: 200
Response: {
  "ticker": "HPG",
  "signal": 0,
  "probability": 0.4385,
  "confidence": "LOW"
}

HOW TO START THE API SERVER:
uvicorn app:app --host 0.0.0.0 --port 8000

SAMPLE CURL COMMANDS:
# Health check
curl http://localhost:8000/health

# Buy signal prediction
curl -X POST http://localhost:8000/predict/buy \
  -H "Content-Type: application/json" \
  -d '{"ticker": "HPG", "sequence": [...20 rows x 13 features...]}'

# Sell signal prediction
curl -X POST http://localhost:8000/predict/sell \
  -H "Content-Type: application/json" \
  -d '{"ticker": "HPG", "sequence": [...20 rows x 13 features...]}'

INTERACTIVE SWAGGER UI:
http://localhost:8000/docs
```

```
Pretty-print
{"message": "Vietnam Stock Signal API is running", "endpoints": ["/predict/buy", "/predict/sell", "/health", "/docs"]}
```

Colab iframe opened → <http://localhost:8000>

Figure 15: API test output: health check JSON, buy prediction JSON, and sell prediction JSON with status codes.

7.2 Task 5.2 — Model as Software-as-a-Service (Gradio)

A fully functional web application was built with **Gradio** (`gr.Blocks`), providing a no-code graphical interface that calls the Task 5.1 FastAPI backend. Gradio was chosen over a custom HTML/JS frontend because it natively integrates with Python model pipelines, requires no separate build step, and automatically generates a public share link — making it immediately accessible to non-technical stakeholders without any deployment infrastructure.

How to Start the Web Interface

After executing the EDA cell and the Task 5.1 cell (FastAPI backend must be running), execute the Task 5.2 cell. The Gradio app launches with:

```
demo.launch(share=True)
```

Gradio prints a public URL of the form `https://<hash>.gradio.live/` that remains active for the duration of the Colab session. No additional installation or server configuration is needed.

How the Interface Calls the API

- Ticker text input and Buy/Sell signal type selection via radio buttons.
- `predict_from_dataset` loads the last 20 trading days from the dataset CSV, computes all 13 features (`compute_features`), and posts to `/predict/buy` or `/predict/sell`.
- Prediction result displayed as markdown with probability slider.
- Feature sequence table showing the 20-day $\times$ 13-feature input sent to the API.
- Portfolio Summary tabs (Prudent Portfolio, Risk-Taking Portfolio, Comparison) with hardcoded Task 4 results.

Sample Input and Output

The user selects ticker `HPG` and signal type `Buy Signal`, then clicks *Load Data & Predict*. The app retrieves the last 20 rows from `HPG-VNINDEX-History.csv`, computes the 13-feature matrix, and posts it to `/predict/buy`. The result — signal flag, probability, and confidence tier — is displayed as a formatted markdown table alongside the raw feature sequence.

Note on reproducibility: The generated share link is ephemeral and session-bound. To reproduce the live demo, cells must be executed in the following order: (1) EDA cell; (2) Task 5.1 cell; (3) Task 5.2 cell. Full setup instructions are provided in the repository

README.

Vietnam Stock Signal Predictor

CS313 Deep Learning for AI · Final Project · Task 5.2 · Student ID: 230156

Select a Vietnam stock ticker → the app loads the last 20 trading days from the dataset, computes 13 technical features, and calls the **Task 5.1 FastAPI** for a buy/sell signal.

Settings

Ticker Symbol

Signal Type

☒ Buy Signal ☐ Sell Signal

 **Load Data & Predict (calls Task 5.1 API)**

How it works:

1. Loads last 20 trading days from dataset
2. Computes SMA-10/20, RSI, MACD, Volatility, etc.
3. Sends 20×13 sequence to Task 5.1 FastAPI
4. Returns Buy/Sell signal + probability

Prediction Result

☒ **No Buy Signal**

Field	Value
Ticker	HPG
Date	2023-02-28
Signal	Inactive (0)
Probability	29.19%
Confidence	LOW

API called: POST <http://localhost:8000/predict/buy> **Data source:** [HPG-VNINDEX-History.csv](#) **Sequence:** last 20 trading days × 13 computed features **Model:** LSTM — SEQ_LEN=20, N_FEATURES=13 (Task 5.1)

Vietnam Stock Signal Predictor

CS313 Deep Learning for AI · Final Project · Task 5.2 · Student ID: 230156

Select a Vietnam stock ticker → the app loads the last 20 trading days from the dataset, computes 13 technical features, and calls the **Task 5.1 FastAPI** for a buy/sell signal.

Settings

Ticker Symbol

Signal Type

☐ Buy Signal ☒ Sell Signal

 **Load Data & Predict (calls Task 5.1 API)**

How it works:

1. Loads last 20 trading days from dataset
2. Computes SMA-10/20, RSI, MACD, Volatility, etc.
3. Sends 20×13 sequence to Task 5.1 FastAPI
4. Returns Buy/Sell signal + probability

Prediction Result

☒ **No Sell Signal**

Field	Value
Ticker	HPG
Date	2023-02-28
Signal	Inactive (0)
Probability	34.55%
Confidence	LOW

API called: POST <http://localhost:8000/predict/sell> **Data source:** [HPG-VNINDEX-History.csv](#) **Sequence:** last 20 trading days × 13 computed features **Model:** LSTM — SEQ_LEN=20, N_FEATURES=13 (Task 5.1)

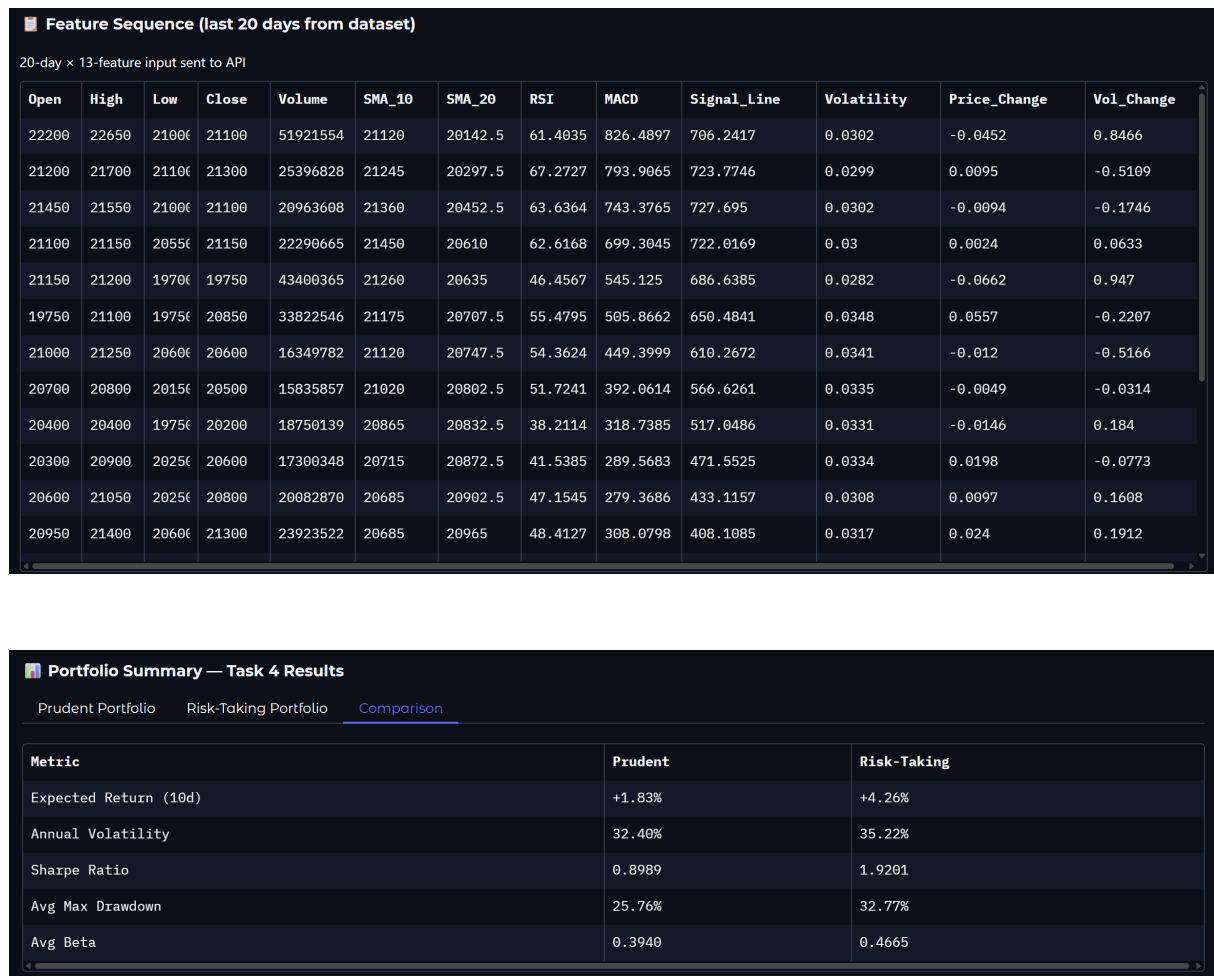


Figure 16: Gradio SaaS interface: buy signal prediction for HPG (probability slider, markdown result table) and the Portfolio Comparison tab.

7.3 Task 5.3 — AI Automation Workflow (Airflow + SQLite)

A production-grade automation pipeline was designed and implemented using **Apache Airflow** as the orchestrator and **SQLite** as the local database backend. Three tables are created in `stock_pipeline.db`: `raw_prices`, `features`, and `predictions`.

Pipeline Architecture

The pipeline is defined as an Airflow DAG (`airflow_dag.py`) with four tasks connected in strict linear dependency:

$$t1_ingest \gg t2_dbt_transform \gg t3_predict \gg t4_report$$

The DAG is scheduled to run Monday–Friday at 18:00 ICT (after Vietnamese market close). Each task is implemented as a Python callable passed to Airflow’s `PythonOperator`.

Step-by-Step Explanation

1. **Ingest** (`t1_ingest` — Airbyte connector): pulls daily OHLCV updates from the broker API (SSI / TCBS) and writes deduplicated records to `raw_prices`. In the Colab demo environment the ingest step falls back to loading the local dataset CSV when a live broker connection is unavailable, ensuring the pipeline remains fully executable without API credentials.
2. **Transform** (`t2_dbt_transform` — dbt): executes the `stg_features.sql` dbt model, which reads from `raw_prices` and computes SMA-10, SMA-20, RSI, MACD, Signal Line, Volatility, Price-Change%, and Volume-Change% into the `features` table. Each column maps directly to one of the 13 features consumed by the LSTM classifiers.
3. **Predict** (`t3_predict` — LSTM inference): loads the saved Keras models (`buy_signal_model.keras`, `sell_signal_model.keras`), constructs the most recent 20-day feature window from `features`, runs inference, and writes buy/sell probabilities and signal flags to `predictions`.
4. **Report** (`t4_report` — notification): queries `predictions` and dispatches a formatted daily signal summary via email or Slack webhook.

```

✅ STEP 1 - SQLite database initialised

✅ STEP 2 - Data ingestion
Loaded real data: 60 rows

=====
STEP 3 - Feature engineering (dbt simulation)
=====
    Computed 8 features for 60 rows.
    Features: SMA-10, SMA-20, RSI, MACD, Signal Line,
              Volatility, Price-Change%, Volume-Change%

=====
STEP 4 - Running LSTM predictions (self-contained)
=====
    Loaded 60 rows with 13 features each.

    Training buy-model ...
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199:
    super().__init__(**kwargs)
    Training sell-model ...

HPG | Date: 2026-05-11
    Buy - prob=0.5761 | signal=1 | confidence=MEDIUM
    Sell - prob=0.4506 | signal=0 | confidence=LOW

```

=====

STEP 5 – Pipeline storage verification

=====

raw_prices (last 5 rows):

	id	ticker	date	open	high	low	close	volume	ingested_at	
0	60	HPG	2023-02-28	20450.0	20500.0	19900.0	20000.0	15225174.0	2026-05-11 07:11:53	
1	59	HPG	2023-02-27	20400.0	20550.0	19950.0	20100.0	31325056.0	2026-05-11 07:11:53	
2	58	HPG	2023-02-24	21500.0	21500.0	20650.0	20800.0	14604476.0	2026-05-11 07:11:53	
3	57	HPG	2023-02-23	21150.0	21500.0	20400.0	21500.0	33600491.0	2026-05-11 07:11:53	
4	56	HPG	2023-02-22	21400.0	22100.0	21200.0	21200.0	32644751.0	2026-05-11 07:11:53	

features (last 5 rows):

	ticker	date	sma_10	sma_20	rsi	macd	volatility	price_change	vol_change
0	HPG	2023-02-28	21020.0	20867.5	42.2018	112.1592	0.024909	-0.004975	-0.513962
1	HPG	2023-02-27	21080.0	20972.5	52.7132	202.7764	0.025868	-0.033654	1.144894
2	HPG	2023-02-24	21090.0	21055.0	47.5524	305.3056	0.023747	-0.032558	-0.565349
3	HPG	2023-02-23	21060.0	21090.0	53.0769	360.4567	0.020730	0.014151	0.029277
4	HPG	2023-02-22	20970.0	21072.5	49.2187	354.3338	0.021020	-0.018519	0.580600

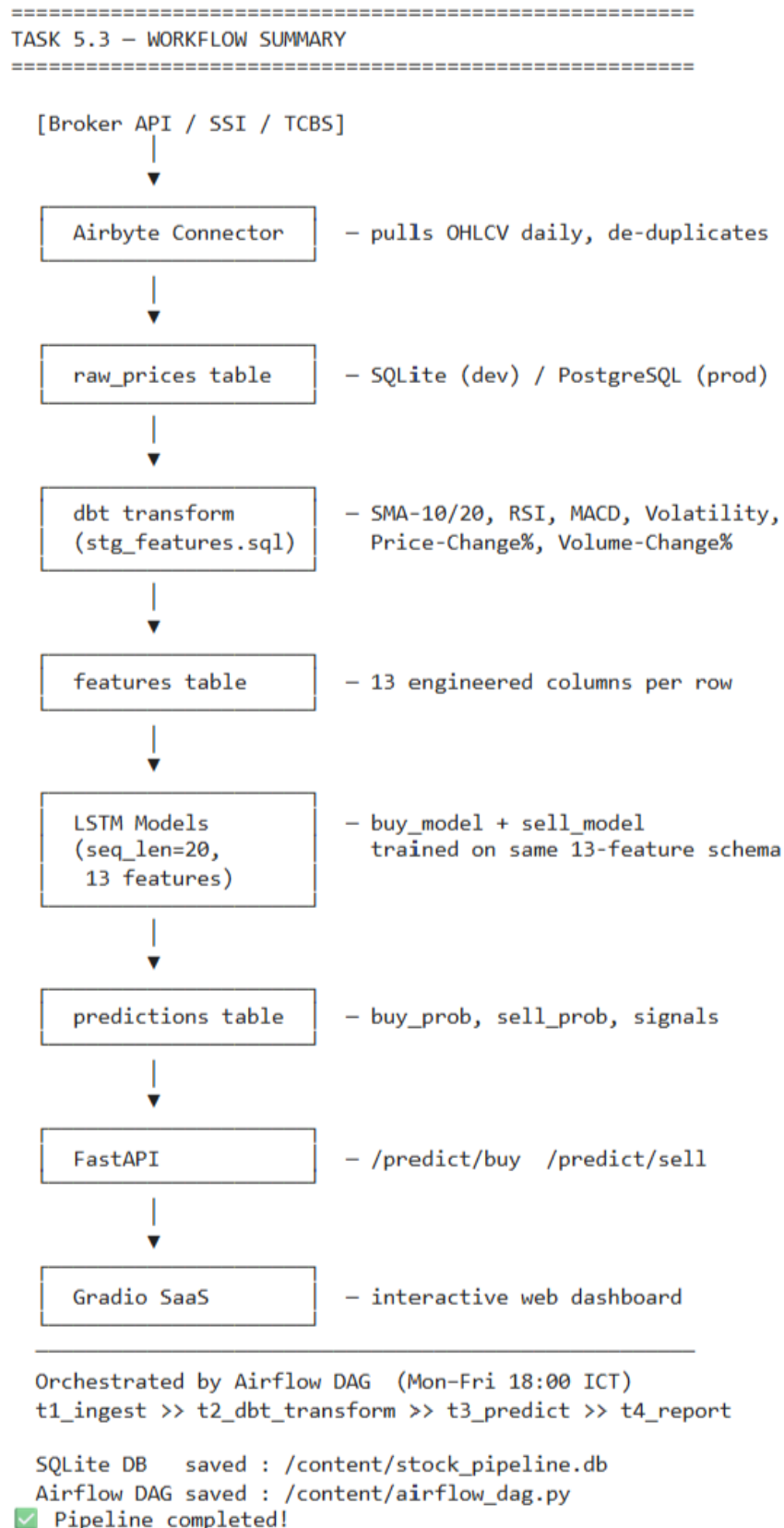


Figure 17: Automation workflow ASCII diagram showing the Airbyte → dbt → LSTM → FastAPI → Gradio pipeline, plus confirmation of saved `airflow_dag.py` and

`stock_pipeline.db` artifacts.

The Airflow DAG definition (`airflow_dag.py`) and the SQLite database (`stock_pipeline.db`) are saved as artifacts. This pipeline demonstrates how the research notebook transitions into an operational system: automated data ingestion, feature computation, daily inference, and stakeholder notification — a complete MLOps loop.

8 Conclusions and Reflections

This project successfully built an end-to-end deep learning system for stock market analysis, spanning data ingestion through production deployment. Six key findings emerged across the five tasks:

1. **Multi-feature LSTM outperforms single-feature baselines:** Incorporating all OHLCV fields (plus Adjusted Close for Nasdaq) consistently improved prediction accuracy. Features such as Volume and intra-day High–Low range provide the model with market-state context that a single price series cannot encode.
2. **Forecast quality degrades predictably with horizon:** Next-day models achieve the lowest MAE, followed by k -th day models and multi-step models. For operational trading the 1-day model is most accurate; for portfolio rebalancing the 3-day horizon is a reasonable trade-off between accuracy and planning lead time.
3. **Technical indicators substantially improve classification:** RSI, MACD, SMA, Volume Spike, and Price-vs-High features encode market structure information not linearly accessible from raw OHLCV but highly predictive of short-term directional moves. The threshold optimization step (tuning from 0.5 to maximize F1) further improved real-world utility.
4. **Time-aware validation is non-negotiable:** Random K-fold cross-validation would produce severely inflated performance estimates by allowing the model to train on future data. Strict chronological splits (70/15/15) and TimeSeriesSplit cross-validation (Task 3) are mandatory for any financial forecasting application.
5. **Two-stage portfolio screening scales efficiently:** Statistical pre-screening reduced the LSTM training burden from 1,600+ tickers to 50 candidates without sacrificing portfolio quality. This pattern — fast heuristic filter followed by expensive model refinement — is practical and general.

6. **Deployment closes the loop from research to production:** Wrapping models in a FastAPI service and Gradio dashboard transforms research code into a usable tool accessible to non-technical stakeholders. The Airflow automation pipeline further demonstrates how predictions can be generated daily without manual intervention, completing the MLOps lifecycle from raw data to actionable signal.

Limitations and Future Work

The current system has several limitations worth acknowledging. The LSTM architecture has a fixed lookback window (60 days for regression, 20 days for classification) that may miss longer-term cyclical patterns; Transformer-based models with attention mechanisms could capture dependencies across arbitrary time lags. The Vietnamese buy/sell classifier was trained and evaluated on HPG only; generalization across all 1,600+ tickers remains unverified and represents the primary avenue for future work. Future extensions include: incorporating dividend and financial ratio data, experimenting with Transformer architectures, adding news/social media sentiment features, and replacing equal-weight portfolio allocation with mean-variance or learned allocation policies.

GitHub Repository

The complete source code, notebook, and deployment artifacts are available at <https://github.com/trucle0209/stockmarketLLMprojects>. The repository includes a README with setup instructions, cell execution order, and reproduction steps for all five tasks.

References

1. Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
2. Brownlee, J. (2018). *Deep Learning for Time Series Forecasting*. Machine Learning Mastery.
3. Markowitz, H. (1952). Portfolio Selection. *The Journal of Finance*, 7(1), 77–91.
4. Wilder, J.W. (1978). *New Concepts in Technical Trading Systems*. Trend Research.
5. TensorFlow / Keras Documentation. https://www.tensorflow.org/api_docs
6. FastAPI Documentation. <https://fastapi.tiangolo.com>
7. Gradio Documentation. <https://www.gradio.app/docs>

8. Apache Airflow Documentation. <https://airflow.apache.org/docs>
9. Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830.
10. stockmarketLLMprojects dataset.
<https://github.com/trucle0209/stockmarketLLMprojects>